

Tugas Praktikum 1: Scraping & Eksplorasi Data (EDA)

Nama: Rayyan Muhtar Ali **NRP:** 5054251051 **Mata Kuliah:** Penambangan Data

Sumber data: halaman *Top Anime* MyAnimeList (<https://myanimelist.net/topanime.php>), diambil pakai `requests` + `BeautifulSoup`.

Datanya tabel peringkat anime. Satu halaman isinya 50 baris, saya ambil 40 halaman, jadi totalnya 2000 baris dengan 7 kolom mentah.

Kenapa pilih sumber ini? Pertama, tabelnya HTML biasa jadi tidak perlu Selenium atau semacamnya. Kedua, kolomnya campur angka dan kategori, jadi ada bahan buat dihitung sekaligus dikelompokkan.

1. Scraping Dataset

Dari tiap baris tabel saya ambil: peringkat, judul, tipe dan jumlah episode, periode tayang, jumlah anggota, skor, sama URL halaman detailnya.

```
In [1]: import re
import time

import requests
import truststore
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from bs4 import BeautifulSoup

truststore.inject_into_ssl() # antivirus di laptop saya bikin SSL error, jadi paka

sns.set_theme(style="whitegrid")
plt.rcParams["figure.figsize"] = (8, 4.5)
pd.set_option("display.width", 140)
pd.set_option("display.max_columns", 20)
```

```
In [2]: HEADERS = {
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 "
    "(KHTML, like Gecko) Chrome/140.0.0.0 Safari/537.36"
}
URL = "https://myanimelist.net/topanime.php?limit={}"

def ambil_baris(html):
    """Ambil semua baris anime dari satu halaman peringkat."""
```

```

sup = BeautifulSoup(html, "html.parser")
hasil = []

for tr in sup.select("tr.ranking-list"):
    tag_judul = tr.select_one("h3.anime_ranking_h3 a")
    info = tr.select_one("div.information").get_text("\n", strip=True).split("\n")
    info = [x.strip() for x in info if x.strip()]

    hasil.append({
        "peringkat": tr.select_one("td.rank span").get_text(strip=True),
        "judul": tag_judul.get_text(strip=True),
        "tipe_episode": info[0] if len(info) > 0 else None,
        "periode_tayang": info[1] if len(info) > 1 else None,
        "anggota": info[2] if len(info) > 2 else None,
        "skor": tr.select_one("td.score span").get_text(strip=True),
        "url": tag_judul["href"],
    })

return hasil

```

```

In [3]: data = []

for limit in range(0, 2000, 50):
    r = requests.get(URL.format(limit), headers=HEADERS, timeout=30)
    r.raise_for_status()
    data += ambil_baris(r.text)

    if limit % 250 == 0:
        print(f"halaman limit={limit:<5} total baris = {len(data)}")

    time.sleep(1.2) # jeda supaya tidak membebani server

print("selesai, total baris:", len(data))

```

```

halaman limit=0      total baris = 50
halaman limit=250    total baris = 300
halaman limit=500    total baris = 550
halaman limit=750    total baris = 800
halaman limit=1000   total baris = 1050
halaman limit=1250   total baris = 1300
halaman limit=1500   total baris = 1550
halaman limit=1750   total baris = 1800
selesai, total baris: 2000

```

```

In [4]: df_mentah = pd.DataFrame(data)
df_mentah.to_csv("data_mentah_anime.csv", index=False, encoding="utf-8")

print(df_mentah.shape)
df_mentah.head()

```

```

(2000, 7)

```

Out[4]:

	peringkat	judul	tipe_episode	periode_tayang	anggota	skor	
0	1	Sousou no Frieren	TV (28 eps)	Sep 2023 - Mar 2024	1,514,421 members	9.25	https://myanimelist
1	2	Fullmetal Alchemist: Brotherhood	TV (64 eps)	Apr 2009 - Jul 2010	3,734,710 members	9.11	https://myanimeli
2	3	Re:Zero kara Hajimeru Isekai Seikatsu 4th Season	TV (19 eps)	Apr 2026 - Sep 2026	328,230 members	9.11	https://myanimelis
3	4	Bleach: Sennen Kessen-hen - Kashin-tan	TV (10 eps)	Jul 2026 - Oct 2026	164,305 members	9.09	https://myanimeli
4	5	Steel Ball Run: JoJo no Kimyou na Bouken	ONA (? eps)	Mar 2026 -	233,438 members	9.08	https://myanime



Hasil mentahnya langsung disimpan ke `data_mentah_anime.csv`. Jadi kalau nanti mau ngulang analisisnya saja, tidak perlu scraping dari awal lagi.

2. Import dan Pemahaman Dataset

```
In [5]: df_mentah = pd.read_csv("data_mentah_anime.csv")
df_mentah.head()
```

Out[5]:	peringkat	judul	tipe_episode	periode_tayang	anggota	skor	
0	1	Sousou no Frieren	TV (28 eps)	Sep 2023 - Mar 2024	1,514,421 members	9.25	https://myanimelist
1	2	Fullmetal Alchemist: Brotherhood	TV (64 eps)	Apr 2009 - Jul 2010	3,734,710 members	9.11	https://myanimeli
2	3	Re:Zero kara Hajimeru Isekai Seikatsu 4th Season	TV (19 eps)	Apr 2026 - Sep 2026	328,230 members	9.11	https://myanimelis
3	4	Bleach: Sennen Kessen-hen - Kashin-tan	TV (10 eps)	Jul 2026 - Oct 2026	164,305 members	9.09	https://myanimeli
4	5	Steel Ball Run: JoJo no Kimyou na Bouken	ONA (? eps)	Mar 2026 -	233,438 members	9.08	https://myanime

```
In [6]: print("Jumlah baris dan kolom:", df_mentah.shape)
print()
df_mentah.info()
```

Jumlah baris dan kolom: (2000, 7)

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2000 entries, 0 to 1999
Data columns (total 7 columns):
#   Column          Non-Null Count  Dtype
---  -
0   peringkat        2000 non-null   int64
1   judul            2000 non-null   object
2   tipe_episode     2000 non-null   object
3   periode_tayang  2000 non-null   object
4   anggota          2000 non-null   object
5   skor             2000 non-null   float64
6   url              2000 non-null   object
dtypes: float64(1), int64(1), object(5)
memory usage: 109.5+ KB
```

```
In [7]: df_mentah.dtypes
```

```
Out[7]: peringkat      int64
        judul          object
        tipe_episode    object
        periode_tayang  object
        anggota         object
        skor            float64
        url             object
        dtype: object
```

Semua kolom masih `object`, soalnya hasil scraping memang berupa teks. Ada juga kolom yang isinya dua informasi sekaligus: `tipe_episode` bentuknya `TV (28 eps)`, `periode_tayang` bentuknya `Sep 2023 - Mar 2024`. Yang seperti ini saya pecah dulu, kalau tidak nanti tidak bisa dihitung apa-apa.

```
In [8]: df = df_mentah.copy()

df["peringkat"] = df["peringkat"].astype(int)
df["skor"] = pd.to_numeric(df["skor"], errors="coerce")
df["jumlah_anggota"] = pd.to_numeric(
    df["anggota"].str.replace(r"^\0-9", "", regex=True), errors="coerce"
)

# "TV (28 eps)" -> tipe = TV, jumlah_episode = 28
pecah = df["tipe_episode"].str.extract(r"^(?P<tipe>.+)s*\((?P<eps>[\d?]+\s)eps\)")
df["tipe"] = pecah["tipe"]
df["jumlah_episode"] = pd.to_numeric(pecah["eps"], errors="coerce")

# "Sep 2023 - Mar 2024" -> bulan dan tahun mulai, lalu tahun selesai
mulai = df["periode_tayang"].str.extract(r"^(?:(?P<bulan>[A-Z][a-z]{2})\s+)?(?P<tahun>[A-Z][a-z]{2})")
df["bulan_mulai"] = mulai["bulan"]
df["tahun_mulai"] = pd.to_numeric(mulai["tahun"], errors="coerce")

selesai = df["periode_tayang"].str.extract(r"-s*(?:[A-Z][a-z]{2}\s+)?(?P<tahun>\d{4})")
df["tahun_selesai"] = pd.to_numeric(selesai["tahun"], errors="coerce")

df["status_tayang"] = np.where(df["tahun_selesai"].isna(), "Masih tayang", "Selesai")
df["dekade"] = df["tahun_mulai"] // 10 * 10

df = df[["peringkat", "judul", "tipe", "jumlah_episode", "skor", "jumlah_anggota",
        "bulan_mulai", "tahun_mulai", "tahun_selesai", "status_tayang", "dekade",
        ]]

df.head()
```

Out[8]:

	peringkat	judul	tipe	jumlah_episode	skor	jumlah_anggota	bulan_mulai	tahu
0	1	Sousou no Frieren	TV	28.0	9.25	1514421	Sep	
1	2	Fullmetal Alchemist: Brotherhood	TV	64.0	9.11	3734710	Apr	
2	3	Re:Zero kara Hajimeru Isekai Seikatsu 4th Season	TV	19.0	9.11	328230	Apr	
3	4	Bleach: Sennen Kessen-hen - Kashin-tan	TV	10.0	9.09	164305	Jul	
4	5	Steel Ball Run: JoJo no Kimyou na Bouken	ONA	NaN	9.08	233438	Mar	

In [9]: `df.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2000 entries, 0 to 1999
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   peringkat              2000 non-null   int64  
1   judul                  2000 non-null   object  
2   tipe                   2000 non-null   object  
3   jumlah_episode         1986 non-null   float64 
4   skor                   2000 non-null   float64 
5   jumlah_anggota         2000 non-null   int64  
6   bulan_mulai            1998 non-null   object  
7   tahun_mulai            2000 non-null   int64  
8   tahun_selesai          1976 non-null   float64 
9   status_tayang          2000 non-null   object  
10  dekade                  2000 non-null   int64  
11  url                     2000 non-null   object  
dtypes: float64(3), int64(4), object(5)
memory usage: 187.6+ KB
```

Identifikasi tipe atribut

Setelah dipecah, kolomnya jadi 12. Ini klasifikasinya:

Kolom	Tipe atribut	Alasan
peringkat	Ordinal	angka urutan; jarak peringkat 1 ke 2 tidak sama artinya dengan 1500 ke 1501
judul	Nominal (identitas)	nama, tidak ada urutannya
tipe	Nominal	TV / Movie / ONA / OVA / Special / TV Special
jumlah_episode	Numerik diskrit	hasil cacahan, tidak mungkin 12,5 episode
skor	Numerik kontinu	rata-rata rating, ada angka di belakang koma
jumlah_anggota	Numerik diskrit	cacahan orang yang masukin anime ke list
bulan_mulai	Nominal (siklis)	nama bulan, tiap tahun berulang lagi
tahun_mulai	Numerik diskrit (interval)	tahun mulai tayang
tahun_selesai	Numerik diskrit (interval)	tahun selesai tayang
status_tayang	Nominal biner	cuma dua nilai: selesai atau masih tayang
dekade	Ordinal	kelompok tahun, jelas ada urutannya
url	Nominal (identitas)	alamat halaman detail

3. Analisis Statistik dan Kualitas Data

```
In [10]: kolom_numerik = ["skor", "jumlah_episode", "jumlah_anggota", "tahun_mulai", "peringkat"]

ringkasan = pd.DataFrame({
    "rata_rata": df[kolom_numerik].mean(),
    "median": df[kolom_numerik].median(),
    "modus": df[kolom_numerik].mode().iloc[0],
    "std": df[kolom_numerik].std(),
    "min": df[kolom_numerik].min(),
    "maks": df[kolom_numerik].max(),
}).round(2)

ringkasan
```

```
Out[10]:
```

	rata_rata	median	modus	std	min	maks
skor	7.96	7.87	7.58	0.34	7.55	9.25
jumlah_episode	17.95	12.00	1.00	53.05	1.00	1787.00
jumlah_anggota	323486.14	124991.50	1143.00	510697.39	864.00	4425476.00
tahun_mulai	2014.13	2016.00	2023.00	9.89	1970.00	2026.00
peringkat	1000.50	1000.50	1.00	577.49	1.00	2000.00

```
In [11]: df[kolom_numerik].describe().round(2)
```

```
Out[11]:
```

	skor	jumlah_episode	jumlah_anggota	tahun_mulai	peringkat
count	2000.00	1986.00	2000.00	2000.00	2000.00
mean	7.96	17.95	323486.14	2014.13	1000.50
std	0.34	53.05	510697.39	9.89	577.49
min	7.55	1.00	864.00	1970.00	1.00
25%	7.69	1.00	42644.25	2009.00	500.75
50%	7.87	12.00	124991.50	2016.00	1000.50
75%	8.17	19.75	358451.50	2022.00	1500.25
max	9.25	1787.00	4425476.00	2026.00	2000.00

Dua hal yang langsung kelihatan di sini. `skor` rentangnya sempit banget, dari 7,55 ke 9,25 saja. Sebaliknya `jumlah_anggota` dan `jumlah_episode` jomplang: rata-ratanya jauh di atas median, yang artinya distribusinya menceng ke kanan dan ditarik sama segelintir nilai besar.

```
In [12]: for kol in ["tipe", "status_tayang", "dekade"]:
          tabel = pd.DataFrame({
              "frekuensi": df[kol].value_counts(dropna=False),
              "persen": (df[kol].value_counts(dropna=False, normalize=True) * 100).round(
          })
          print(f"--- {kol} ---")
          print(tabel)
          print()
```



```

--- tipe ---
           frekuensi  persen
tipe
TV           1031    51.55
Movie        451    22.55
ONA          219    10.95
OVA          167     8.35
Special       81     4.05
TV Special   51     2.55

--- status_tayang ---
           frekuensi  persen
status_tayang
Selesai      1976    98.8
Masih tayang   24     1.2

--- dekade ---
           frekuensi  persen
dekade
2010          756    37.80
2020          716    35.80
2000          353    17.65
1990          112     5.60
1980           45     2.25
1970           18     0.90

```

```

In [13]: print("Sepuluh bulan mulai tayang terbanyak")
df["bulan_mulai"].value_counts().head(10)

```

Sepuluh bulan mulai tayang terbanyak

```

Out[13]: bulan_mulai
Apr      405
Oct      375
Jul       293
Jan       258
Dec       108
Mar       106
Sep        95
Aug        93
Feb        79
Jun        67
Name: count, dtype: int64

```

Nilai yang hilang

```

In [14]: hilang = pd.DataFrame({
    "jumlah_hilang": df.isna().sum(),
    "persen": (df.isna().mean() * 100).round(2),
})

hilang

```

Out[14]:

	jumlah_hilang	persen
peringkat	0	0.0
judul	0	0.0
tipe	0	0.0
jumlah_episode	14	0.7
skor	0	0.0
jumlah_anggota	0	0.0
bulan_mulai	2	0.1
tahun_mulai	0	0.0
tahun_selesai	24	1.2
status_tayang	0	0.0
dekade	0	0.0
url	0	0.0

```
In [15]: df[df["jumlah_episode"].isna()].head()
```

Out[15]:

	peringkat	judul	tipe	jumlah_episode	skor	jumlah_anggota	bulan_mulai	tahun
	4	5	Steel Ball Run: JoJo no Kimyou na Bouken	ONA	NaN	9.08	233438	Mar
	55	56	One Piece	TV	NaN	8.73	2726660	Oct
	98	99	Chiikawa	TV	NaN	8.62	25635	Apr
	183	184	Tian Guan Cifu Short Films	ONA	NaN	8.47	12486	Feb
	321	322	Mushen Ji	ONA	NaN	8.31	16399	Oct

```
In [16]: df[df["bulan_mulai"].isna()][["judul", "tipe", "tahun_mulai", "tahun_selesai", "sta
```

Out[16]:

	judul	tipe	tahun_mulai	tahun_selesai	status_tayang
1466	Saint Seiya: Meiou Hades Juuni Kyuu-hen - Yomi...	Special	2003	2003.0	Selesai
1681	Meitantei Conan Movie 10: Tantei-tachi no Requ...	TV Special	2006	2006.0	Selesai

Nilai hilangnya sedikit, dan setelah saya cek satu-satu penyebabnya jelas, bukan gara-gara scraping-nya gagal:

- `jumlah_episode` kosong di anime yang masih tayang, di situsnya memang ditulis `?` eps .
- `tahun_selesai` kosong karena formatnya `Apr 2026 -` , belum ada tanggal akhirnya.
- `bulan_mulai` kosong di dua judul lama yang di MyAnimeList cuma dicatat tahunnya.

Jadi saya tidak mengisinya dengan rata-rata atau median. Nilainya bukan hilang secara acak, tapi memang belum ada. Kalau jumlah episode anime yang belum tamat saya tambal pakai median, datanya justru jadi salah. Barisnya tetap saya pakai, cuma dilewati di perhitungan yang butuh kolom itu. Kebetulan kolom `status_tayang` yang saya bikin juga sekaligus jadi penanda baris mana yang datanya belum lengkap.

Deteksi outlier

Metodenya IQR. Nilai yang jatuh di luar $Q1 - 1,5 \times IQR$ sampai $Q3 + 1,5 \times IQR$ dihitung outlier.

```
In [17]: def batas_iqr(s):
    q1, q3 = s.quantile([0.25, 0.75])
    iqr = q3 - q1
    return q1 - 1.5 * iqr, q3 + 1.5 * iqr

baris = []
for kol in ["skor", "jumlah_episode", "jumlah_anggota"]:
    nilai = df[kol].dropna()
    bawah, atas = batas_iqr(nilai)
    n = int(((nilai < bawah) | (nilai > atas)).sum())
    baris.append({
        "kolom": kol,
        "batas_bawah": round(bawah, 2),
        "batas_atas": round(atas, 2),
        "jumlah_outlier": n,
        "persen": round(n / len(nilai) * 100, 2),
    })

pd.DataFrame(baris)
```

Out[17]:

	kolom	batas_bawah	batas_atas	jumlah_outlier	persen
0	skor	6.97	8.89	27	1.35
1	jumlah_episode	-27.12	47.88	142	7.15
2	jumlah_anggota	-431066.62	832162.38	221	11.05

```
In [18]: fig, ax = plt.subplots(1, 3, figsize=(13, 4))

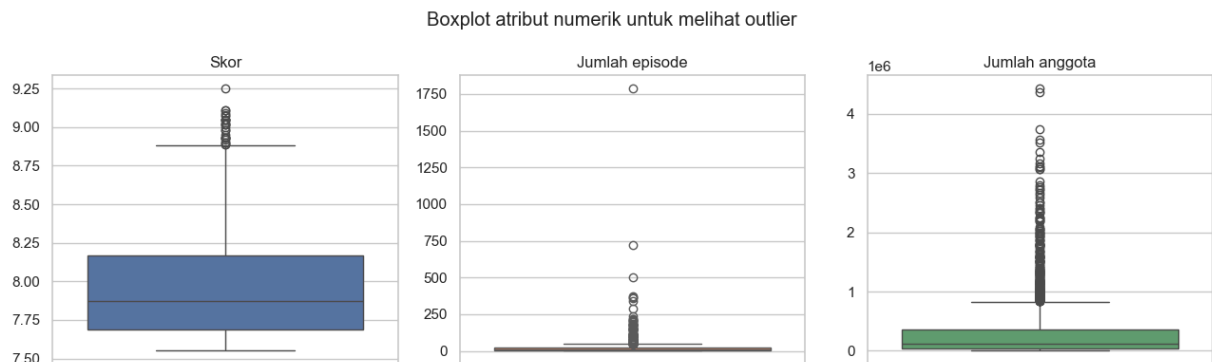
sns.boxplot(y=df["skor"], ax=ax[0], color="#4c72b0")
ax[0].set_title("Skor")

sns.boxplot(y=df["jumlah_episode"], ax=ax[1], color="#dd8452")
ax[1].set_title("Jumlah episode")

sns.boxplot(y=df["jumlah_anggota"], ax=ax[2], color="#55a868")
ax[2].set_title("Jumlah anggota")

for a in ax:
    a.set_ylabel("")

fig.suptitle("Boxplot atribut numerik untuk melihat outlier")
plt.tight_layout()
plt.show()
```



```
In [19]: print("Episode terbanyak")
print(df.nlargest(5, "jumlah_episode")[["judul", "tipe", "jumlah_episode", "skor"]])
print()
print("Anggota terbanyak")
print(df.nlargest(5, "jumlah_anggota")[["judul", "tipe", "jumlah_anggota", "skor"]])
```

Episode terbanyak

	judul	tipe	jumlah_episode	skor
	Doraemon (1979)	TV	1787.0	7.90
	Wushen Zhuzai	ONA	720.0	7.59
	Naruto: Shippuuden	TV	500.0	8.29
Kochira Katsushikaku Kameari Kouenmae	Hashutsujo	TV	373.0	7.82
	Bleach	TV	366.0	8.00

Anggota terbanyak

	judul	tipe	jumlah_anggota	skor
	Shingeki no Kyojin	TV	4425476	8.58
	Death Note	TV	4363546	8.62
Fullmetal Alchemist: Brotherhood		TV	3734710	9.11
	One Punch Man	TV	3558606	8.47
	Kimetsu no Yaiba	TV	3507154	8.39

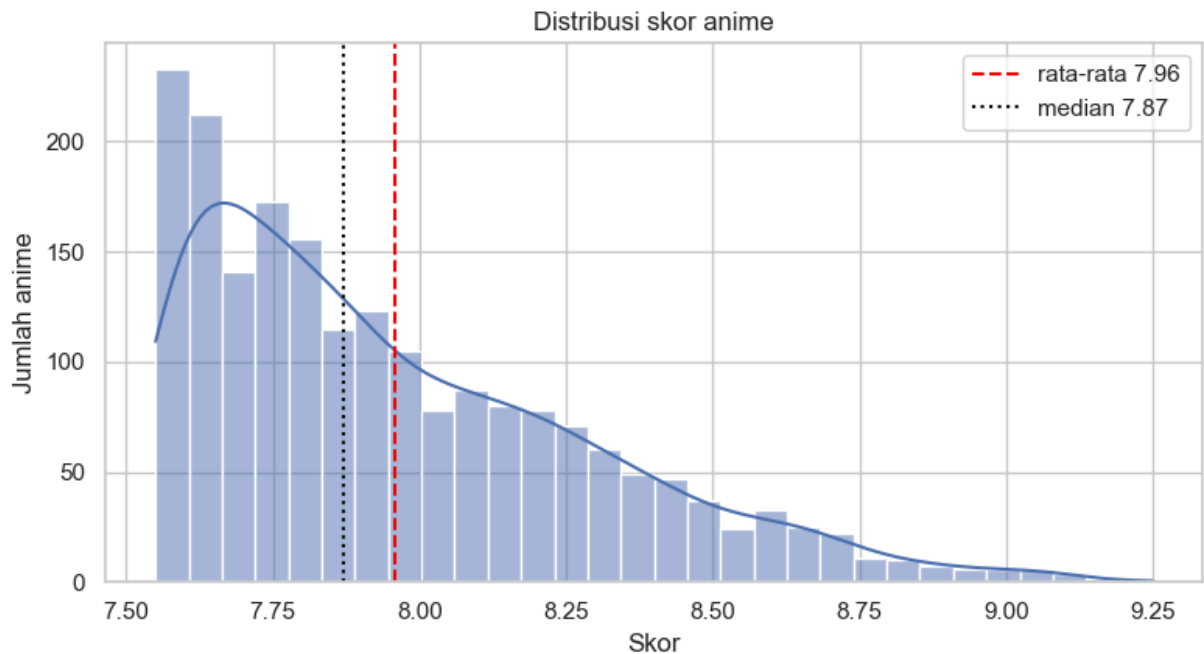
Kalau dilihat isinya, outlier ini nilainya masuk akal semua, bukan salah input. Doraemon (1979) memang benar 1787 episode karena tayang puluhan tahun. *Shingeki no Kyojin* memang dimasukin list sama 4,4 juta orang, sementara median dataset ini cuma sekitar 125 ribu. Outlier di skor malah muncul justru karena datanya terlalu seragam, jadi begitu ada judul yang skornya 9 ke atas langsung lewat batas IQR.

Karena itu outliernya saya biarkan, tidak dibuang dan tidak dipotong. Yang saya ubah cuma cara menggambaranya: jumlah_anggota dipakai skala logaritmik supaya bentuk distribusinya masih kebaca dan tidak ketarik sama judul-judul populer tadi.

4. Visualisasi Data

4.1 Distribusi skor

```
In [20]: fig, ax = plt.subplots()
sns.histplot(df["skor"], bins=30, kde=True, color="#4c72b0", ax=ax)
ax.axvline(df["skor"].mean(), color="red", linestyle="--", label=f"rata-rata {df['skor'].nunique()}")
ax.axvline(df["skor"].median(), color="black", linestyle=":", label=f"median {df['skor'].nunique()}")
ax.set_title("Distribusi skor anime")
ax.set_xlabel("Skor")
ax.set_ylabel("Jumlah anime")
ax.legend()
plt.tight_layout()
plt.show()
```



Skornya numpuk di 7,6-7,9 dan menceng ke kanan, makin tinggi skornya makin sedikit judulnya. Rata-ratanya 7,96, medianya 7,87, jadi ekor atas menarik rata-rata sedikit tapi tidak banyak.

Satu catatan penting: ini bukan gambaran skor semua anime, cuma 2000 peringkat teratas. Makanya sisi kirinya kepotong di 7,55.

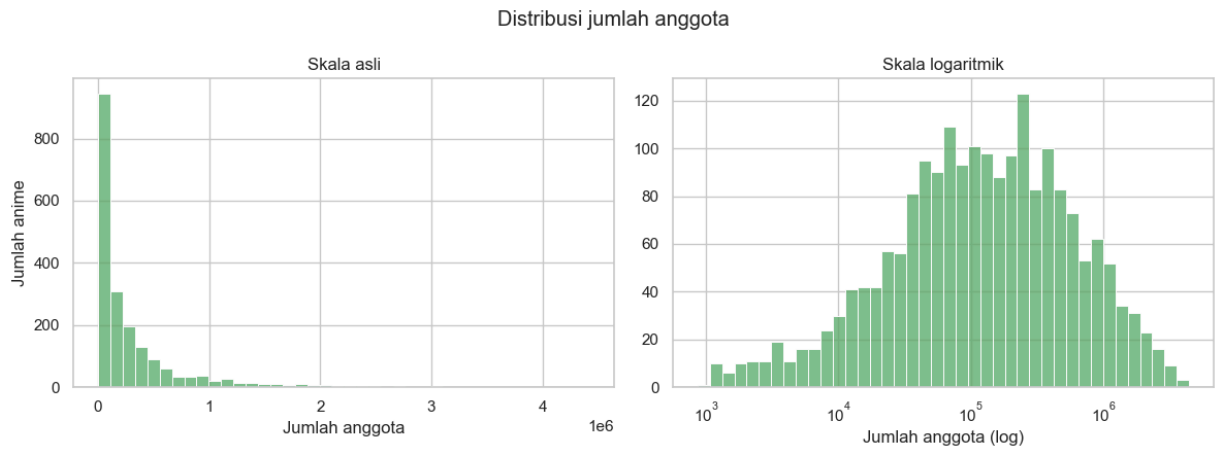
4.2 Distribusi jumlah anggota

```
In [21]: fig, ax = plt.subplots(1, 2, figsize=(12, 4.5))

sns.histplot(df["jumlah_anggota"], bins=40, color="#55a868", ax=ax[0])
ax[0].set_title("Skala asli")
ax[0].set_xlabel("Jumlah anggota")
ax[0].set_ylabel("Jumlah anime")

sns.histplot(df["jumlah_anggota"], bins=40, log_scale=True, color="#55a868", ax=ax[1])
ax[1].set_title("Skala logaritmik")
ax[1].set_xlabel("Jumlah anggota (log)")
ax[1].set_ylabel("")

fig.suptitle("Distribusi jumlah anggota")
plt.tight_layout()
plt.show()
```

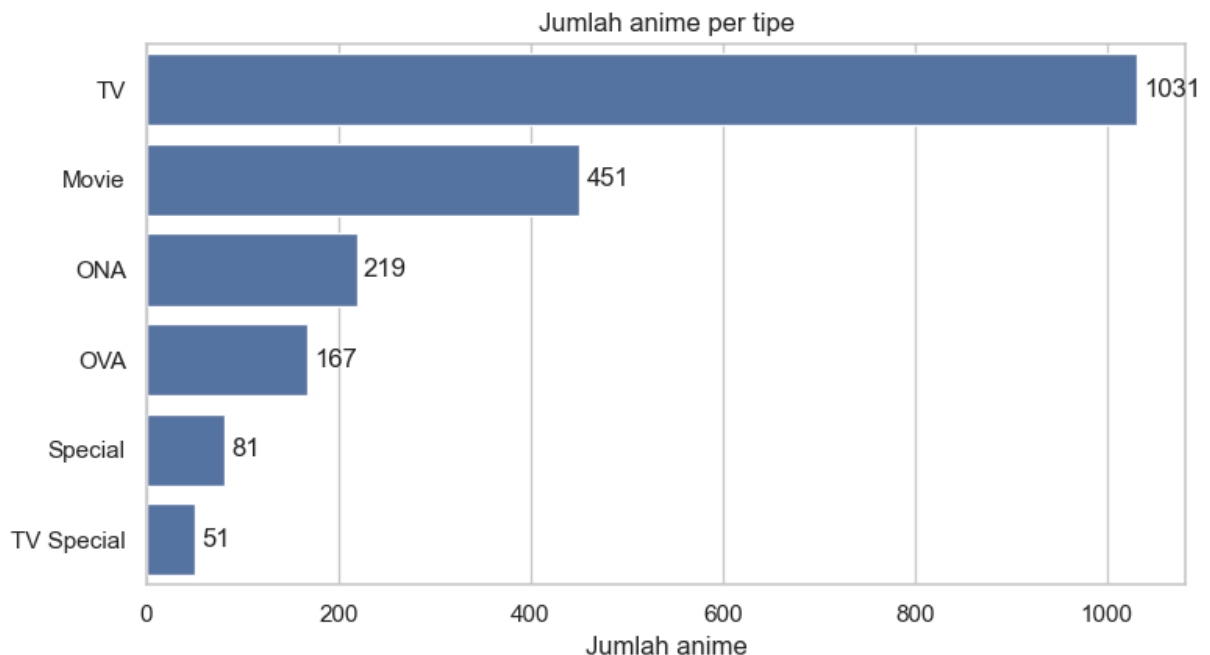


Di skala asli, batangnya numpuk semua di kiri gara-gara ada judul yang anggotanya jutaan. Begitu dipindah ke skala log, bentuknya berubah jadi mendekati simetris. Artinya popularitas anime itu log-normal: banyak judul yang anggotanya puluhan sampai ratusan ribu, dan cuma sedikit yang benar-benar masif.

4.3 Jumlah anime per tipe

```
In [22]: urutan = df["tipe"].value_counts().index
```

```
fig, ax = plt.subplots()
sns.countplot(data=df, y="tipe", order=urutan, color="#4c72b0", ax=ax)
ax.bar_label(ax.containers[0], padding=3)
ax.set_title("Jumlah anime per tipe")
ax.set_xlabel("Jumlah anime")
ax.set_ylabel("")
plt.tight_layout()
plt.show()
```

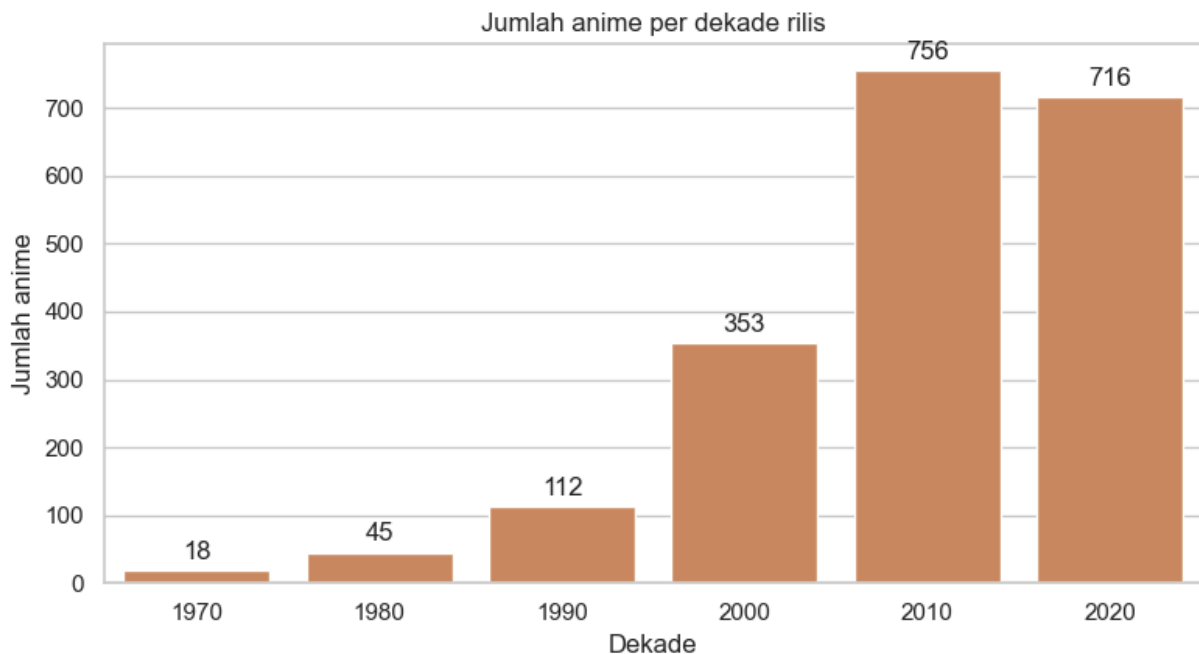


TV paling banyak, lalu Movie. Sisanya (ONA, OVA, Special, TV Special) jumlahnya jauh lebih kecil. Masuk akal sih, format panjang seperti serial TV dan film punya lebih banyak ruang buat bikin ceritanya bagus, sedangkan Special biasanya cuma episode tambahan jadi jarang menembus 2000 besar.

4.4 Jumlah anime per dekade

```
In [23]: per_dekade = df["dekade"].value_counts().sort_index()

fig, ax = plt.subplots()
sns.barplot(x=per_dekade.index.astype(int).astype(str), y=per_dekade.values, color=
ax.bar_label(ax.containers[0], padding=3)
ax.set_title("Jumlah anime per dekade rilis")
ax.set_xlabel("Dekade")
ax.set_ylabel("Jumlah anime")
plt.tight_layout()
plt.show()
```



Paling banyak dari 2010-an (756 judul). Yang menarik, 2020-an sudah 716 padahal dekadanya belum genap sepuluh tahun. Anime sebelum 1990-an sedikit sekali, digabung semua cuma 63 judul.

Jadi dataset ini condong ke anime modern. Wajar, karena yang memberi rating di MyAnimeList itu pengguna aktif, dan mereka kebanyakan nonton judul baru.

4.5 Hubungan skor dan jumlah anggota

```
In [24]: fig, ax = plt.subplots(figsize=(8.5, 5))
sns.scatterplot(data=df, x="jumlah_anggota", y="skor", hue="tipe", alpha=0.6, s=25,
ax.set_xscale("log"))
```

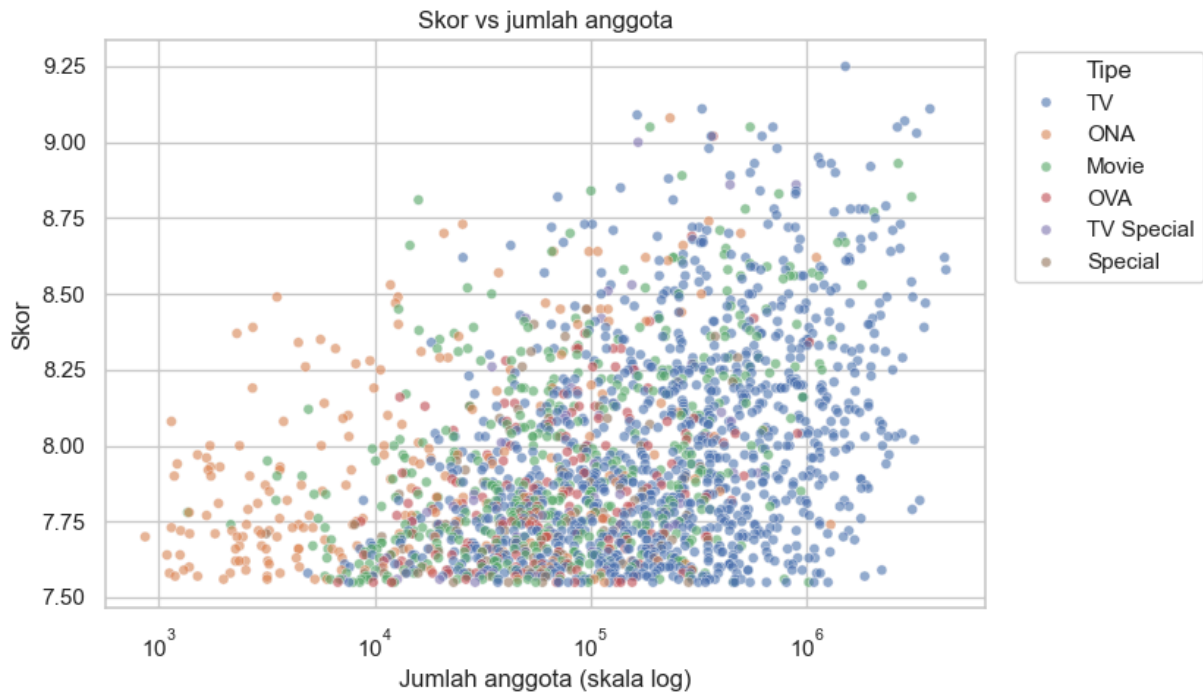


```

ax.set_title("Skor vs jumlah anggota")
ax.set_xlabel("Jumlah anggota (skala log)")
ax.set_ylabel("Skor")
ax.legend(title="Tipe", bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout()
plt.show()

print("Korelasi Pearson skor vs log10(anggota):",
      round(df["skor"].corr(np.log10(df["jumlah_anggota"])), 3))

```



Korelasi Pearson skor vs log10(anggota): 0.39

Trennya naik, tapi tidak kuat. Korelasi skor dengan log jumlah anggota cuma 0,39 dan titiknya menyebar lebar. Jadi anime populer cenderung skornya lebih tinggi, tapi populer bukan berarti otomatis bagus.

Yang juga kelihatan, ada lumayan banyak titik di kiri atas: anggotanya sedikit tapi skornya tinggi. Ini biasanya judul niche yang penontonnya terbatas tapi yang nonton suka. Titik Movie juga kebanyakan ngumpul di kiri, median anggotanya sekitar 72 ribu, jauh di bawah TV yang 260 ribu.

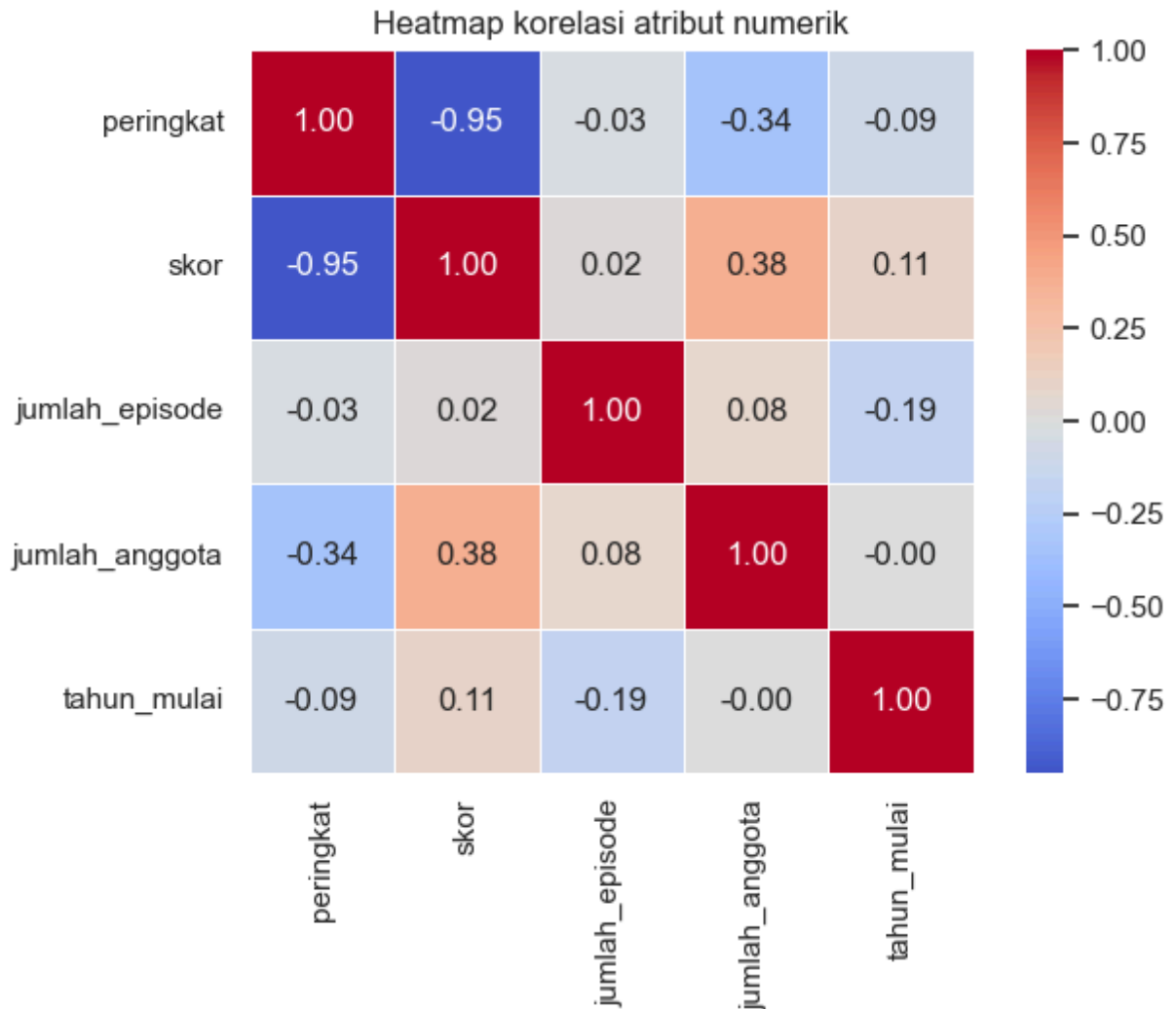
4.6 Korelasi antar atribut numerik

```

In [25]: korelasi = df[["peringkat", "skor", "jumlah_episode", "jumlah_anggota", "tahun_mula"]

fig, ax = plt.subplots(figsize=(7, 5.5))
sns.heatmap(korelasi, annot=True, fmt=".2f", cmap="coolwarm", center=0, square=True,
            linewidths=0.5, ax=ax)
ax.set_title("Heatmap korelasi atribut numerik")
plt.tight_layout()
plt.show()

```



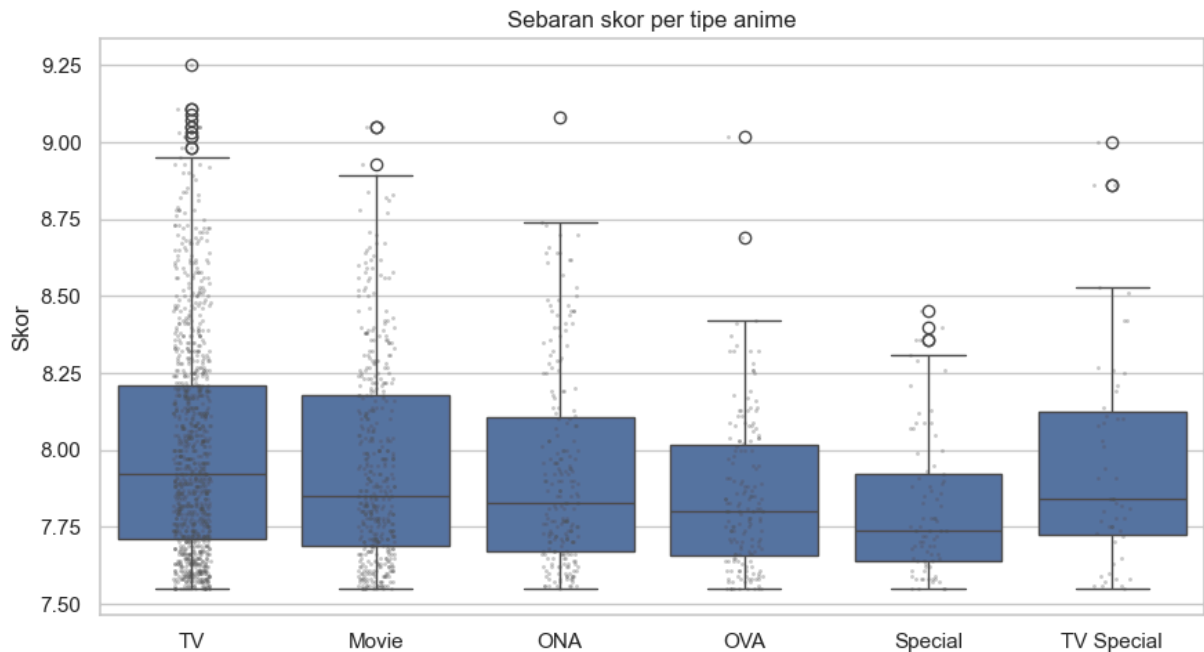
peringkat dan skor korelasinya -0,95, hampir sempurna. Ini wajar karena peringkat MyAnimeList memang disusun dari skor, jadi dua kolom ini isinya informasi yang sama. Kalau nanti dipakai buat modeling, salah satunya harus dibuang.

Sisanya kecil-kecil saja: jumlah episode dengan skor cuma 0,02, tahun rilis dengan skor 0,11. Artinya anime yang panjang atau yang baru rilis tidak otomatis skornya lebih tinggi. Yang lumayan menonjol tetap skor dengan jumlah anggota (0,38).

4.7 Sebaran skor per tipe

```
In [26]: fig, ax = plt.subplots(figsize=(9, 5))
sns.boxplot(data=df, x="tipe", y="skor", order=urutan, ax=ax)
sns.stripplot(data=df, x="tipe", y="skor", order=urutan, size=2, color=".3", alpha=
ax.set_title("Sebaran skor per tipe anime")
ax.set_xlabel("")
ax.set_ylabel("Skor")
plt.tight_layout()
plt.show()

df.groupby("tipe")["skor"].agg(["count", "mean", "median", "std"]).round(2).sort_va
```



Out[26]:

	count	mean	median	std
tipe				
TV	1031	8.00	7.92	0.35
Movie	451	7.95	7.85	0.33
TV Special	51	7.96	7.84	0.36
ONA	219	7.93	7.83	0.32
OVA	167	7.85	7.80	0.25
Special	81	7.82	7.74	0.23

Median tiap tipe berdekatan, cuma beda dari 7,74 (Special) sampai 7,92 (TV). Jadi tipe anime bukan pembeda skor yang berarti. TV dan Movie ekor atasnya lebih panjang, tapi itu karena jumlah judulnya paling banyak jadi peluang muncul judul bagus juga lebih besar. OVA dan Special paling rapat sebarannya, standar deviasinya 0,25 dan 0,23.

```
In [27]: df.to_csv("data_bersih_anime.csv", index=False, encoding="utf-8")
print("data bersih disimpan:", df.shape)
```

data bersih disimpan: (2000, 12)

5. Laporan Analisis

Deskripsi Dataset

Dataset diambil dengan scraping 40 halaman peringkat *Top Anime* MyAnimeList, hasilnya **2000 baris dan 7 kolom mentah**. Setelah kolom gabungan dipecah jadi **12 kolom**, jadi ketentuan minimal 5 fitur dan 1500 baris sudah terpenuhi.

Kolomnya campur beberapa tipe atribut: nominal (`judul` , `tipe` , `bulan_mulai` , `status_tayang` , `url`), ordinal (`peringkat` , `dekade`), numerik diskrit (`jumlah_episode` , `jumlah_anggota` , `tahun_mulai` , `tahun_selesai`), dan numerik kontinu (`skor`). Tahunnya dari 1970 sampai 2026. Tipenya ada enam, dengan TV paling banyak (1031 judul, 51,6%), lalu Movie (451), ONA (219), OVA (167), Special (81), dan TV Special (51). Dari 2000 judul, 1976 sudah selesai tayang dan 24 masih jalan.

Statistik Deskriptif

Atribut	Rata-rata	Median	Modus	Std	Min	Maks
skor	7,96	7,87	7,58	0,34	7,55	9,25
jumlah_episode	17,95	12	1	53,05	1	1787
jumlah_anggota	323.486	124.992	1.143	510.697	864	4.425.476
tahun_mulai	2014	2016	2023	9,89	1970	2026

Yang paling kelihatan dari tabel ini, `skor` sebarannya sempit sekali karena isinya memang cuma peringkat teratas. Sebaliknya `jumlah_episode` dan `jumlah_anggota` rata-ratanya jauh di atas median, tanda distribusinya menceng ke kanan. Standar deviasi `jumlah_anggota` bahkan lebih besar dari rata-ratanya sendiri, jadi datanya benar-benar menyebar.

Untuk kolom kategorikal, `tipe` didominasi TV (51,6%) dan Movie (22,6%). Dari `bulan_mulai` , April (405) dan Oktober (375) paling sering muncul. Ini cocok dengan kebiasaan industri anime yang merilis serial barunya di awal musim semi dan musim gugur.

Penanganan Data

Nilai hilang cuma ada di tiga kolom dan semuanya di bawah 2%: `tahun_selesai` 24 baris (1,2%), `jumlah_episode` 14 baris (0,7%), dan `bulan_mulai` 2 baris (0,1%). Setelah dicek satu per satu, ternyata bukan scraping-nya yang gagal, tapi nilainya memang belum ada. Anime yang masih tayang ditulis `? eps` tanpa tanggal akhir, dan dua judul lama di MyAnimeList cuma dicatat tahunnya saja.

Karena itu tidak saya isi pakai rata-rata atau median. Nilai yang hilangnya bukan acak, jadi menambal jumlah episode anime yang belum tamat justru memalsukan data. Barisnya tetap saya pakai, cuma dilewati di perhitungan yang butuh kolom tersebut. Kolom `status_tayang` sekaligus jadi penanda baris mana yang belum lengkap.

Outliernya dideteksi pakai IQR. Hasilnya `jumlah_anggota` 221 outlier (11,05%), `jumlah_episode` 142 outlier (7,15%), dan `skor` 27 outlier (1,35%). Angka-angkanya valid semua: Doraemon (1979) memang 1787 episode, dan *Shingeki no Kyojin* memang masuk list 4,4 juta orang. Outlier `skor` muncul justru karena datanya kelewat seragam, jadi judul yang skornya di atas 8,89 langsung dianggap ekstrem. Karena itu outliernya tidak dibuang

maupun dipotong. Yang disesuaikan cuma visualisasinya, `jumlah_anggota` digambar pakai skala log supaya bentuk sebarannya masih kebaca.

Visualisasi dan Interpretasi

Ada tujuh grafik di bagian 4. Ringkasan temuannya:

1. **Histogram skor** menceng kanan dan numpuk di 7,6-7,9. Rentangnya sempit karena dataset cuma berisi peringkat teratas.
2. **Histogram jumlah anggota** numpuk di kiri pada skala asli, tapi jadi mendekati simetris di skala log. Popularitas anime polanya log-normal.
3. **Bar chart tipe** menunjukkan TV dan Movie mendominasi, sementara format pendek seperti Special jarang masuk 2000 besar.
4. **Bar chart per dekade** memperlihatkan dataset condong ke anime 2010-an dan 2020-an.
5. **Scatter plot skor vs jumlah anggota** korelasinya 0,39 di skala log. Positif tapi lemah sampai sedang, dan banyak judul niche yang anggotanya sedikit tapi skornya tinggi.
6. **Heatmap korelasi** menegaskan `peringkat` dan `skor` praktis redundan (-0,95), sedangkan jumlah episode dan tahun rilis hampir tidak ada hubungannya dengan skor.
7. **Boxplot skor per tipe** menunjukkan median antar tipe cuma beda tipis, jadi tipe anime bukan pembeda kualitas.

Kesimpulan

Hasil scraping ini datanya bersih, nilai hilangnya sedikit, dan outliernya nyata sehingga tidak perlu dibuang. Temuan utamanya, skor di peringkat teratas MyAnimeList seragam sekali dan tidak banyak dipengaruhi tipe anime, panjang episode, maupun tahun rilis. Yang paling berkaitan dengan skor adalah popularitas, itu pun cuma lemah sampai sedang.

Kalau dataset ini mau dipakai untuk pemodelan, `peringkat` sebaiknya dibuang karena redundan dengan `skor`, dan `jumlah_anggota` ditransformasi log dulu biar sebarannya tidak terlalu menceng.